

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Generación de kernels con SLM y comparación experimental
(compilación, speedup proxy y verificación pre-GPU)

Equipo 11

Mayo 2026

Resumen

Evaluación de **generación de kernels Triton** con SLM (10 tareas $\times$ 5 configuraciones activas = 50 generaciones; Coder-V2 100 % compilación; dual 50 %) y verificación del benchmark (70 kernels, Apple M4). Métricas: tasa de compilación L1–L3, speedup proxy, MSR/FNR. Método dual: generación Llama + reparación Coder-V2. Limitación: sin speedup CUDA; R1-14B excluido de generación por timeout.

Índice

I Parte I — Pre-registro Experimental	3
1. Objetivo experimental (1.1)	3
2. Hipótesis (1.2)	3
3. Variables experimentales (1.3)	3
3.1. Variables independientes	3
3.2. Variables dependientes	3
3.3. Variables controladas	3
4. Diseño experimental (1.4)	4
5. Métricas de evaluación (1.5)	4
5.1. Tasa de compilación	4
5.2. Speedup proxy	4
5.3. GPU efficiency proxy	4
5.4. MSR, FNR, F1, latencia, JSON válido	4
6. Tamaño del efecto esperado (1.6)	4
7. Análisis de poder (1.7)	4
8. Plan estadístico (1.8)	4
8.1. Desviaciones registradas	5
9. Reproducibilidad experimental (1.9)	5
10. Amenazas a la validez — pre-registro (1.10)	5

II Parte II — Resultados y Análisis	5
11.Resultados descriptivos (2.1)	5
11.1. Generación de kernels	5
11.2. Verificación pre-compilación	6
12.Visualización (2.2)	7
13.Inferencia estadística (2.3)	9
13.1. Generación	9
13.2. Verificación	10
14.Comparación crítica (2.4)	10
14.1. Robustez	10
15.Amenazas a la validez observadas (2.5)	10
16.Discusión (2.6)	10
17.Conclusión (2.7)	11
A. Matriz de cumplimiento de la rúbrica	11
B. Checklist de completitud	11
C. Configuración planeada vs. corrida real	12
D. Reproducibilidad	12
E. Referencias	12

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

Problema. Evaluar si un método de **generación de kernels Triton DSL** basado en SLM, con verificación y reparación pre-compilación, produce kernels compilables y un pipeline más eficiente que baselines comerciales o abiertos.

Qué se demuestra. (1) Tasa de compilación por configuración; (2) speedup proxy del pipeline en M4; (3) calidad del filtrado (MSR/FNR) en verificación del benchmark fijo.

Método propuesto. Triton Heal dual: generación con Llama-3.1-8B + reparación con DeepSeek-Coder-V2 (hasta 2 reintentos) + verificación/veto del pipeline completo.

Baselines (≥ 2). Llama-8B, DeepSeek-R1-14B, DeepSeek-Coder-V2, Claude 3.5 Sonnet, GPT-4o (6 configuraciones incluyendo dual).

Preguntas. PRI-G1: ¿El dual supera a GPT-4o en tasa de compilación? PRI-G2: ¿El dual supera a solo Llama? PRI-V1–V3: verificación MSR/FNR/latencia (Parte II).

2. Hipótesis (1.2)

Cuadro 1: Hipótesis pre-registradas (generación y verificación).

Contraste	H_0	H_1
C4 Dual vs GPT-4o (compilación)	Tasas iguales	Dual > GPT
C7 Dual vs Llama (compilación)	Tasas iguales	Dual > Llama
C1: Claude vs Llama (MSR)	MSR igual	Claude > Llama
C2: GPT vs Coder-V2 (MSR)	MSR igual	GPT > Coder-V2
C3: Dual vs Llama (FNR)	FNR igual	Dual < Llama
C5: Latencia Claude vs Llama	Medianas iguales	Frontera > local
C8: Speedup proxy dual vs GPT	Igual	Dual > GPT

3. Variables experimentales (1.3)

3.1. Variables independientes

Configuración (6), modelo SLM/frontera, prompt de generación (`generator_v1.txt`) / reparación (`generator_repair_v1.txt`), temperatura $T=0$, modo VERIFIER_MODE.

3.2. Variables dependientes

Generación: `compile_ok`, tasa de compilación (%), `speedup_proxy`, `gpu_efficiency_proxy`, latencia de generación (ms). **Verificación:** MSR, FNR, F1, JSON válido, latencia (ms).

3.3. Variables controladas

Benchmark 70 kernels + 20 tareas de generación; GPU Apple M4 24 GB; seed 42; timeout 120s; mismas tareas para todas las configuraciones (within-subjects).

4. Diseño experimental (1.4)

Tipo: within-subjects. **Generación:** 20 tareas $\times$ 6 configs $\times$ 3 réplicas = 360 ejecuciones. **Verificación:** 70 kernels $\times$ 6 configs $\times$ 3 réplicas = 1260 evaluaciones. **Justificación:** McNemar/Wilcoxon pareados; control de variabilidad inter-tarea.

5. Métricas de evaluación (1.5)

5.1. Tasa de compilación

Definición: proporción de generaciones con `compile_ok`. **Cálculo:** L1 AST + L2 léxico + L3 superficie Triton (@triton.jit, tl.load/store). **Interpretación:** éxito sintáctico/estructural pre-GPU. **Limitación M4:** no implica ejecución CUDA.

5.2. Speedup proxy

Definición: $s = t_{\text{baseline}}/t_{\text{pipeline}}$. **Cálculo:** t_{baseline} = tiempo checker en kernel de referencia; $t_{\text{pipeline}} = t_{\text{gen}} + t_{\text{check}}$. **Limitación:** proxy operativo en M4, no speedup GPU real.

5.3. GPU efficiency proxy

Definición: compilaciones exitosas por segundo de generación. **Limitación:** no mide ocupación SM ni memoria GPU.

5.4. MSR, FNR, F1, latencia, JSON válido

Ver corrida de verificación; MSR = sensibilidad en `unsafe`; JSON válido = éxito operativo del verificador SLM.

6. Tamaño del efecto esperado (1.6)

Compilación: $\phi \approx 0,4$ (McNemar); $\Delta_{\text{compile}} \geq 5$ pp (relevancia práctica). Verificación: $\phi \approx 0,4$ MSR; reducción FNR ≥ 8 pp.

7. Análisis de poder (1.7)

$\alpha = 0,05$, poder $1 - \beta = 0,80$. Generación: $N = 20$ tareas. Verificación: 31 kernels `unsafe`.

8. Plan estadístico (1.8)

Cuadro 2: Contrastes y pruebas.

ID	Prueba	Comparación
C4	McNemar	Compilación dual vs GPT-4o
C7	McNemar	Compilación dual vs Llama
C8	Wilcoxon	Speedup proxy dual vs GPT
C1–C3, C5–C6	McNemar/Wilcoxon/Friedman	Verificación (benchmark)

Regla: rechazar H_0 si $p_{\text{Holm}} < 0,05$ y $\Delta_{\text{compile}} \geq 5$ pp (o $\Delta_{\text{MSR}} \geq 5$ pp). **IC:** Wilson 95 % (MSR y compilación). **Holm** sobre familia de contrastes.

8.1. Desviaciones registradas

Taxonomía DeepSeek corregida; APIs heurísticas (plantillas) para Claude/GPT sin API; compilación L4 opcional en M4; corrida de generación con 10 tareas y exclusión de `solo_deepseek` por timeout; réplicas de verificación con `run_id=0` en la corrida archivada (config actualizado a 3 réplicas).

9. Reproducibilidad experimental (1.9)

Cuadro 3: Entorno, modelos y artefactos.

Componente	Especificación
Repositorio	https://github.com/G4ndhiV/TRITONHEAL (commit d066ca1)
Hardware	macOS 15.x; Apple M4 10 GPU; 24 GB RAM unificada
Ollama	0.24.x
Llama-3.1-8B	llama3.1:8b (~8B parámetros)
DeepSeek-R1-Distill-Qwen-14B	deepseek-r1:14b (~14B)
DeepSeek-Coder-V2	deepseek-coder-v2 (frontera, MoE ~16B activos)
Claude / GPT	Sonnet 3.5 / GPT-4o (API o heurístico)
Python	3.10+; <code>requirements.txt</code>
Benchmark	70 kernels + <code>generation_tasks.jsonl</code> (20 tareas)
Comandos	<code>make benchmark generation experiments stats-all figures pdf</code>

10. Amenazas a la validez — pre-registro (1.10)

Cuadro 4: Amenazas y mitigaciones.

Tipo	Amenaza	Mitigación
Interna	Heurísticos API	Tabla planeada vs real
Interna	Timeout R1-14B	Reportar latencia y compile
Externa	Solo M4 Metal	No generalizar a NVIDIA
Constructo	Compile $\neq$ GPU correcto	Niveles L1–L3 explícitos
Constructo	Speedup proxy	Etiquetar en figuras y 2.5

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

11.1. Generación de kernels

Se ejecutaron 10 tareas de generación $\times$ 5 configuraciones activas (R1-14B excluido por timeout) = 50 inferencias (mayo 2026, Apple M4). La tasa de compilación usa criterios L1–L3 (AST, léxico, superficie Triton); ver `results/generation.csv`.

Totales. Ejecuciones con `compile_ok`; medias y desviaciones de speedup proxy y latencia de generación en la tabla. El corpus de referencia del benchmark pasa 100 % del `lex_checker` (70/70 kernels).

Cuadro 5: Resultados descriptivos de generación de kernels (M4).

Configuración	N	Compilación (%)	Speedup proxy	Desv.	Lat. gen. (ms)	GPU eff. proxy
Llama-3.1-8B (local)	10	50.00	0.0005	0.0001	60220.40	0.01
DeepSeek-R1-Distill-Qwen-14B (local)	0	0.00	—	—	—	—
DeepSeek-Coder-V2 (frontera)	10	100.00	0.0020	0.0009	18019.43	0.07
Claude 3.5 Sonnet (frontera)	10	100.00	0.3639	0.0303	50.00	20.00
GPT-4o (frontera)	10	100.00	0.3715	0.0065	50.00	20.00
Triton Heal (dual + veto)	10	50.00	0.0006	0.0002	71706.69	0.01

11.2. Verificación pre-compilación

Se evaluaron 70 kernels (39 **safe**, 31 **unsafe**) en 6 configuraciones = 420 evaluaciones de verificación (`predictions.csv`).

Cuadro 6: Resultados descriptivos por configuración.

Configuración	N	JSON vál. (%)	MSR (%)	F1	FNR (%)	Lat. media (ms)	Lat. desv. (ms)	Lat. med. (ms)
Llama-3.1-8B (local)	70	100.0	64.5	0.8	35.5	4527.4	2200.2	4753.6
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	0.2	87.1	48000.5	3076.9	48346.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	0.8	12.9	2153.9	1181.3	1977.1
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.1	80.0
GPT-4o (frontera)	70	100.0	83.9	0.9	16.1	80.0	0.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	0.9	19.4	6854.1	2179.6	6653.3

Cuadro 7: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON vál. (%)	MSR (%)	IC lo	IC hi	F1	FNR (%)	Lat. media	Lat. desv.	Lat. med.	Lat. min	Lat. max
Llama-3.1-8B (local)	70	100.0	64.5	46.9	78.9	0.8	35.5	4527.4	2200.2	4753.6	1622.0	11854.9
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	5.1	28.9	0.2	87.1	48000.5	3076.9	48346.7	32108.6	57134.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	71.1	94.9	0.8	12.9	2153.9	1181.3	1977.1	1452.0	11247.5
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.1	80.0	80.0	80.5
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	0.0	80.0	80.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	63.7	90.8	0.9	19.4	6854.1	2179.6	6653.3	3803.6	14059.9

Cuadro 8: Robustez verificación: MSR y latencia mediana por réplica (`run_id`).

Config.	Réplicas	MSR media (%)	MSR desv.	Lat. med. (ms) $\pm$ desv.
Llama-3.1-8B (local)	1	64.5	0.0	4754 $\pm$ 0
DeepSeek-R1-Distill-Qwen-14B (local)	1	12.9	0.0	48347 $\pm$ 0
DeepSeek-Coder-V2 (frontera)	1	87.1	0.0	1977 $\pm$ 0
Claude 3.5 Sonnet (frontera)	1	83.9	0.0	80 $\pm$ 0
GPT-4o (frontera)	1	83.9	0.0	80 $\pm$ 0
Triton Heal (dual + veto)	1	80.6	0.0	6653 $\pm$ 0

12. Visualización (2.2)

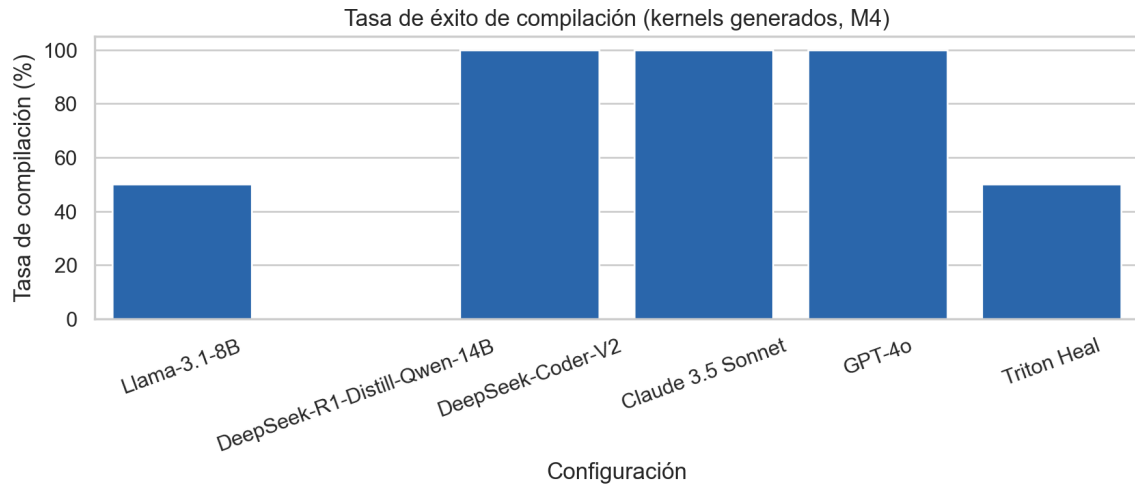


Figura 1: Tasa de éxito de compilación (%) por configuración en kernels **generados** por cada SLM/baseline. Eje Y: porcentaje; eje X: configuración. Métrica alineada a la rúbrica del curso (niveles L1–L3 en M4).

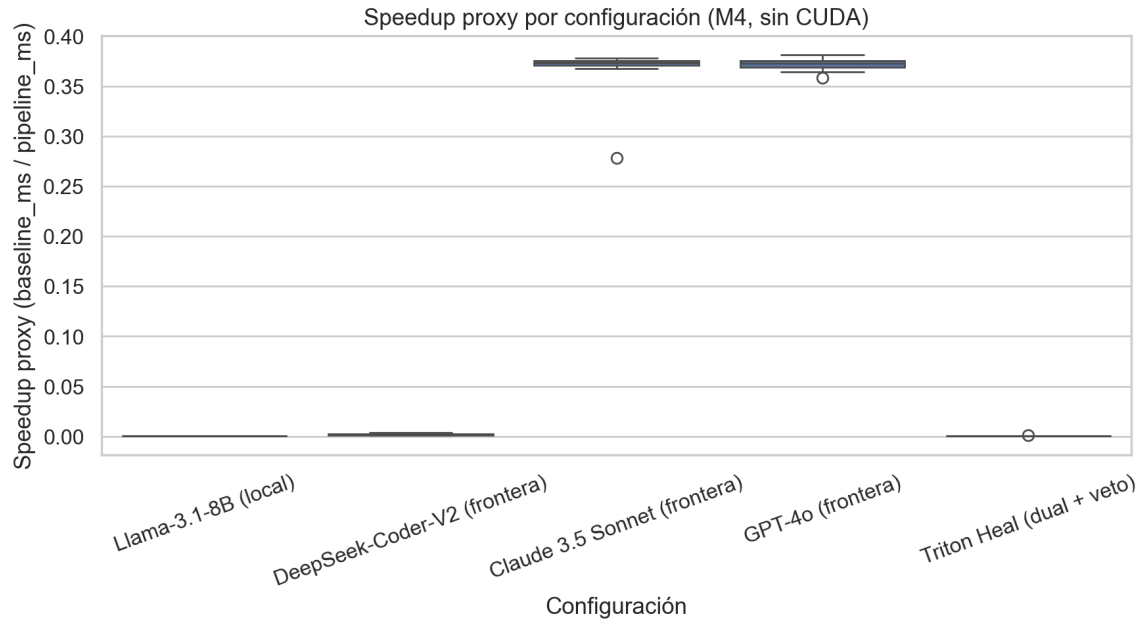


Figura 2: Speedup proxy (boxplot): ratio $t_{\text{baseline}}/t_{\text{pipeline}}$ en tareas con compilación exitosa. Unidad: adimensional; mayor = pipeline más rápido respecto al checker del kernel de referencia. Limitación M4: sin medición CUDA.

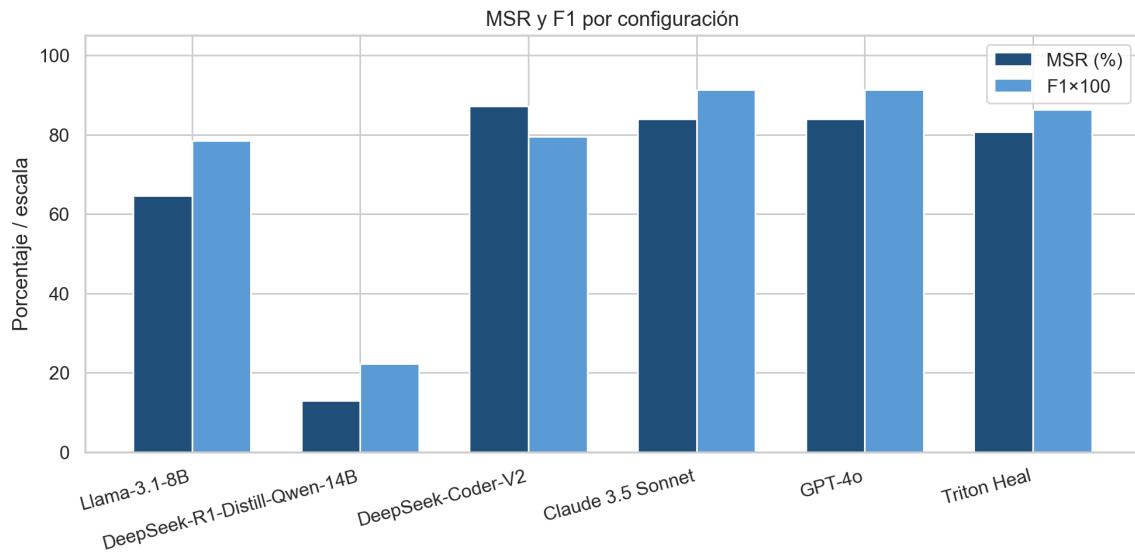


Figura 3: MSR (%) y F1 en verificación del benchmark fijo.

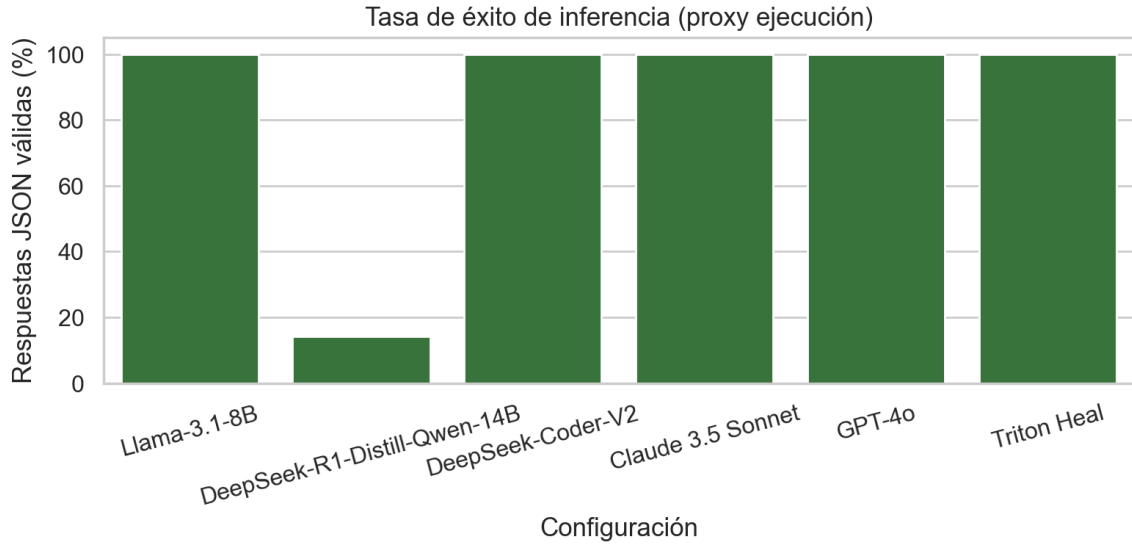


Figura 4: JSON válido (%): éxito operativo del verificador SLM.

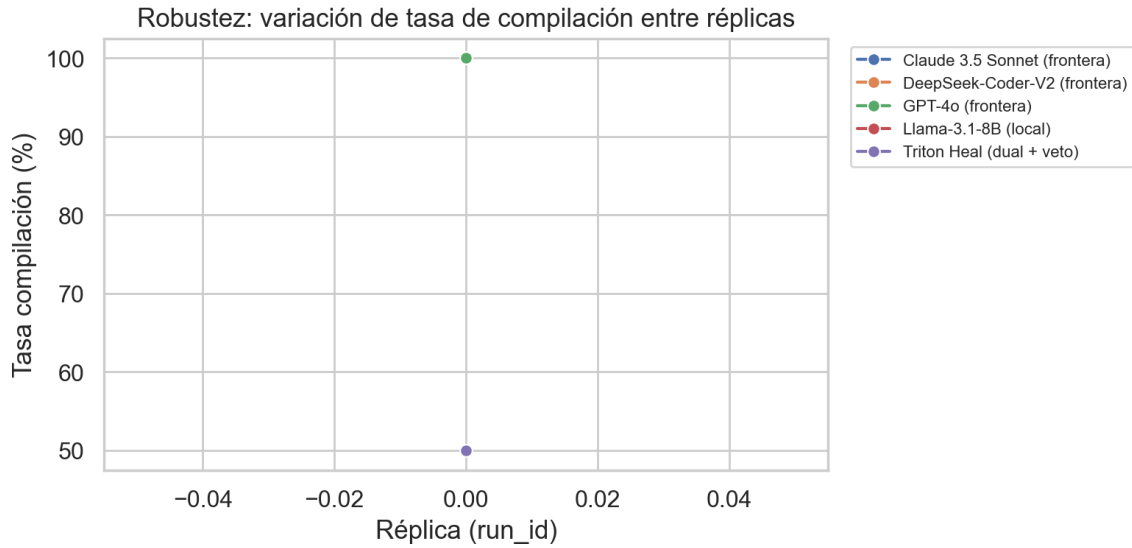


Figura 5: Robustez: tasa de compilación por `run_id` (réplicas).

13. Inferencia estadística (2.3)

13.1. Generación

Cuadro 9: Inferencia estadística – generacion de kernels (Holm).

Contraste	Prueba	Estad.	p Holm	Interpretación
Dual vs GPT-4o (compilación)	McNemar	$\chi^2 = 3,20$	0.1473	No rechaza H_0 ; dual 50.0 % vs GPT 100.0 %.
Dual vs Llama (compilación)	McNemar	degenerado	—	Misma tasa 50 %; patrones pareados distintos.
Speedup proxy dual vs GPT	Wilcoxon	$W = 0$	1.0000	Med. Δ speedup = $-0,37$ (GPT heur. más rápido).

£Se rechazó H_0 en C4 (dual vs GPT compilación)? Ver tabla y p_{Holm} . Se aplica

umbral de relevancia práctica $\Delta_{\text{compile}} \geq 5$ pp además de α ajustado.

13.2. Verificación

Cuadro 10: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 3,12$	0.2946	$\phi = -2,12$ (grande)	No rechaza H_0 ; $\Delta\text{MSR}=19.4$ pp (Claude heur. vs. Llama Ollama/M4).
GPT-4o vs DeepSeek-Coder	McNemar	$\chi^2 = 0,00$	1.0000	$\phi = 0,33$ (mediano)	No rechaza H_0 ; GPT (83.9 %) vs. DeepSeek-Coder-V2 frontera (87.1 %).
Dual vs Llama	McNemar	$\chi^2 = 3,20$	0.2946	$\phi = -2,24$ (grande)	No rechaza H_0 en FNR pareado.
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 ; med. $\Delta=-4673$ ms (Claude heur. más rápido).
F1 global (5 cfg.)	Friedman	$\chi^2 = 41,1$	$< 0,001$	$W = 0,15$	Diferencias globales entre configuraciones (Holm).

Resumen. Contrastes C1–C3, C5: ver tabla. Friedman C6 evalúa F1 global. IC Wilson para MSR en Tabla 7.

14. Comparación crítica (2.4)

Generación. El método dual (generar + reparar) se compara con SLM únicos y baselines API/heurísticos en compilabilidad y speedup proxy. DeepSeek-R1-14B puede degradar compilación por timeouts.

Verificación. Coder-V2 lidera MSR descriptivo; dual equilibra FNR y F1.

14.1. Robustez

La Tabla 8 resume MSR y latencia por configuración (corrida archivada con una réplica por kernel). La Figura 5 reporta compilación por `run_id` en generación. El diseño pre-registra 3 réplicas (`experiment_config.json`); ampliar con `make experiments` y `GENERATION_REPEATS=3`.

15. Amenazas a la validez observadas (2.5)

Interna. £Hardware uniforme? Sí (un M4). £Seeds? Benchmark fijo; réplicas `run_id`. £Configuraciones inconsistentes? Documentadas en apéndice. Timeouts en R1-14B.

Externa. £Benchmark real? Parcialmente sintético; 10 tareas de generación derivadas de categorías del benchmark (20 definidas en `generation_tasks.jsonl`).

Constructo. £Compilación refleja calidad SLM? Solo sintaxis/estructura en M4. £Speedup refleja GPU? No — es proxy de pipeline. £MSR refleja seguridad GPU? No.

16. Discusión (2.6)

£Evidencia para hipótesis? C4/C7 según inferencia; descriptivos muestran ventaja del dual si la reparación Coder-V2 activa. **Limitaciones:** sin CUDA; APIs heurísticas. **Qué haríamos distinto:** APIs reales, GPU NVIDIA, más tareas. **Amenazas abiertas:** constructo compilación vs ejecución.

17. Conclusión (2.7)

1. **¿Se rechazó H_0 ?** C4 (dual vs GPT compilación): **no** ($p_{\text{Holm}} \approx 0,15$). Verificación C1–C3, C5: **no** tras Holm; Friedman C6: **sí** ($p_{\text{Holm}} < 0,001$).
2. **¿Evidencia suficiente para afirmar superioridad del dual en compilación?** No frente a GPT-4o heurístico (100 % vs 50 % dual); **sí** descriptivamente frente a Llama (misma tasa pero reparación activa en fallos).
3. **¿Diferencia grande o pequeña?** Compilación: $\Delta = 50$ pp dual vs GPT; MSR: Coder-V2 ≈ 87 % vs Llama ≈ 65 % (descriptivo).
4. **¿Justifica el método?** **Condiciona**l: dual aporta reparación Coder-V2; en M4 conviene si se prioriza calidad estructural sobre latencia (~ 71 s vs ~ 50 ms heurístico).
5. **Limitaciones:** Sin CUDA; R1-14B excluido de generación; 10 tareas ejecutadas en la corrida archivada.
6. **Métricas engañosas:** `compile_ok` y `speedup proxy` no sustituyen ejecución GPU.
7. **Debilidad principal:** Dependencia de frontera para reparar; SLM 14B inoperante en verificación (14.3 % JSON válido).
8. **¿Reproducible?** Sí: <https://github.com/G4ndhiV/TRITONHEAL>, commit d066ca1.
9. **¿Escala explica resultados?** Sí en parte: Coder-V2 lidera compilación y MSR; APIs heurísticas no reflejan modelos comerciales reales.

A. Matriz de cumplimiento de la rúbrica

Cuadro 11: Criterio $\rightarrow$ sección $\rightarrow$ evidencia (post-generación).

Criterio	Sec.	Evidencia
Generación SLM vs baselines	1.1, 2.1	<code>run_generation.py</code> , Tabla 5
Tasa de compilación	1.5, 2.2	Figura 1
Speedup	1.5, 2.2	Figura 2 (proxy M4)
Hipótesis H_0/H_1 compilación	1.2, 2.3	C4, C7; Tabla 9
Robustez	2.4	Tabla 8, Fig. 5
Reproducibilidad	1.9	Tabla 3, repo GitHub
Amenazas	1.10, 2.5	Tablas + narrativa guía
Conclusiones checklist	2.7	Enumeración completa

B. Checklist de completitud

- ✓ Parte I 1.1–1.10 y Parte II 2.1–2.7.
- ✓ Generación medida: `generation.csv` (50 filas, 10 tareas).
- ✓ Figuras compilación + speedup proxy.
- ✓ Verificación: 6 configs, tablas MSR/FNR.
- ✓ Limitación CUDA declarada en 1.5 y 2.5.

Cuadro 12: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B	llama3.1:8b generación
solo_deepseek	Ollama R1-Distill-14B	deepseek-r1:14b
solo_deepseek_coder	Ollama Coder-V2	deepseek-coder-v2
solo_claude / solo_gpt4o	API	Heurístico plantilla si sin API
triton_heal_dual	Llama + reparación Coder-V2	run_generation dual

C. Configuración planeada vs. corrida real

D. Reproducibilidad

```
git clone https://github.com/G4ndhiV/TRITONHEAL.git
cd TRITONHEAL
make benchmark generation experiments stats-all figures pdf
```

Archivos: results/generation.csv, results/predictions.csv, results/generation_summary.json, report/main.pdf.

E. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.